

RepoSage — Codebase Q&A; with RAG + AST Parsing

■■ ENDGAME TIER — LLM Systems Engineering

"LLM-Powered Repository Intelligence via Abstract Syntax Tree Chunking"

■■ Endgame Project | Domain: RAG + LLM Engineering + AST | Difficulty: Advanced | Timeline: 10 Weeks

■ Project Overview

Description	An LLM-powered codebase Q&A; assistant that ingests entire GitHub repositories and answers natural language questions about the code. Unlike naive RAG systems that chunk by character count, RepoSage uses Abstract Syntax Tree (AST) parsing to chunk code semantically — by function, class, and module — producing dramatically more accurate and context-rich answers.
Target Audience	LLM engineers, backend developers, ML engineers interested in RAG systems
Domain	Retrieval-Augmented Generation, LLM Engineering, AST Parsing, Vector Search
Deliverable	Web-based codebase Q&A; chatbot + AST chunking pipeline + vector retrieval engine
Why It Stands Out	Naive RAG cuts code mid-function, destroying context. AST-based chunking preserves semantic units, making RepoSage answer code questions at a level that basic RAG chatbots fundamentally cannot match.

■■ Tech Stack

Language	Python 3.10+
----------	--------------

AST Parsing	tree-sitter (multi-language), Python ast module
LLM	Claude claude-sonnet-4-20250514 via Anthropic API (or GPT-4o)
Embeddings	OpenAI text-embedding-3-small or sentence-transformers (local)
Vector Store	ChromaDB (local) or Pinecone (cloud)
RAG Framework	LangChain or LlamaIndex (optional orchestration)
GitHub Integration	PyGithub / git clone via subprocess
Backend API	FastAPI
Frontend	Streamlit or React (chat UI)
Dev Tools	Jupyter, VSCode, Git, Docker

■ MVP Features (Phase 1)

Repository Ingestion Pipeline

- ◆ Clone any public GitHub repo via URL using git clone subprocess
- ◆ Recursive file traversal: collect all .py, .js, .ts, .java, .go source files
- ◆ Filter out non-code files: node_modules, .git, build artifacts, binaries
- ◆ Compute file-level metadata: language, line count, last modified, author (git log)

AST-Based Semantic Chunking

- ◆ Parse Python files using Python ast module: extract all functions and classes as independent chunks
- ◆ Parse JS/TS/Java/Go files using tree-sitter universal parser
- ◆ Each chunk carries metadata: file path, function/class name, line range, docstring, imports
- ◆ Preserve parent context: class name attached to method chunks as metadata
- ◆ Fallback: naive character chunking for non-parseable files (config, markdown)

Embedding & Vector Indexing

- ◆ Embed each AST chunk with code-optimised embeddings (text-embedding-3-small or code-specific model)
- ◆ Store vectors + metadata in ChromaDB with collection per repository
- ◆ Implement hybrid search: dense vector search + BM25 sparse keyword search
- ◆ Metadata filtering: filter retrieval by file path, function name, or language

RAG Query Engine

- ◆ Semantic search: embed user query and retrieve top-K relevant code chunks
- ◆ Contextual reranking: rerank retrieved chunks by relevance to query intent
- ◆ Prompt construction: inject retrieved chunks + metadata into structured LLM prompt
- ◆ Multi-turn conversation: maintain chat history with sliding context window

Chat Interface

- ◆ Streamlit or React chat UI with message history
- ◆ Show retrieved source chunks as expandable references below each answer
- ◆ File path + line number links for each cited code chunk
- ◆ Repo ingestion progress bar (file count, chunks created, vectors stored)

■ Development Roadmap

Week 1 Project Setup & Repo Ingestion	■ Set up Python environment, install tree-sitter, ChromaDB, LangChain ■ Build GitHub repo cloner: accepts URL, clones to temp directory ■ Build recursive file traverser with language detection by extension ■ Filter non-code files and generate file inventory manifest
Week 2 Python AST Chunker	■ Build Python AST parser using ast module ■ Extract all top-level functions as independent chunks with docstrings ■ Extract all classes and their methods as separate chunks with class context ■ Attach metadata: file_path, function_name, class_name, start_line, end_line, docstring
Week 3 Multi-Language AST Chunker (tree-sitter)	■ Install tree-sitter and language grammars (JS, TS, Java, Go) ■ Build tree-sitter chunker for JavaScript/TypeScript (extract functions, classes, arrow functions) ■ Build tree-sitter chunker for Java (extract methods, classes, interfaces) ■ Build generic fallback chunker for unsupported languages (character-based with overlap)
Week 4 Embedding Pipeline & ChromaDB Indexing	■ Set up ChromaDB as local vector store with one collection per repo ■ Embed all AST chunks via OpenAI embeddings API (batched for efficiency) ■ Store embeddings + full metadata (file, function, class, line range) in ChromaDB ■ Build incremental indexing: skip already-indexed files using content hash
Week 5 Hybrid Retrieval Engine	■ Build dense vector retrieval: top-K similar chunks via cosine similarity ■ Build BM25 sparse keyword retrieval for exact identifier matching ■ Implement Reciprocal Rank Fusion to merge dense + sparse results ■ Build metadata-filtered retrieval: query specific file or function by name
Week 6 RAG Query Engine & LLM Integration	■ Build query embedder: embed user question, retrieve top-5 chunks ■ Build structured prompt template: system context + retrieved chunks + user question ■ Integrate Claude or GPT-4o API for answer generation ■ Add source citation: LLM response references which chunks it used
Week 7 Multi-Turn Conversation & Memory	■ Implement conversation history with sliding window (last 5 turns) ■ Add follow-up question handling: re-rank retrieved chunks using conversation context ■ Add clarification detection: if query is ambiguous, ask for file name or function scope ■ Build chat session persistence: save/load conversation history per repo

Week 8 FastAPI Backend

- Build FastAPI endpoints: POST /ingest (repo URL), POST /query (question + session_id), GET /sessions
- Add background task processing for repo ingestion (non-blocking)
- Add ingestion status endpoint: progress, file count, chunks created
- Add rate limiting and error handling for all endpoints

Week 9 Streamlit Chat UI

- Build chat interface with message history rendering
- Add repo URL input and ingestion progress bar
- Show expandable source chunk references below each LLM answer
- Add metadata display: file path, function name, line numbers per cited chunk

Week 10 Evaluation, Testing & Deploy

- Build RAG evaluation suite: 20 question-answer pairs for 2 test repos
- Measure retrieval precision@K and answer faithfulness (using RAGAS framework)
- Compare AST chunking vs naive character chunking on same eval set
- Deploy FastAPI on Railway/Render, Streamlit on Streamlit Cloud
- Write README with architecture diagram, AST chunking rationale, and eval results

■ Phase 2 — Future Enhancements

- ◆ Support for monorepos with multiple services and languages
- ◆ Code graph construction: function call graph + dependency graph via AST edges
- ◆ PR review assistant: summarise changes in a pull request and flag risky modifications
- ◆ Bug localisation: given a bug description, retrieve the most likely faulty code chunks
- ◆ Code generation mode: generate new functions that match the style of the indexed repo
- ◆ Private repo support with GitHub OAuth and user-level access control

■ Potential Challenges & Solutions

Challenge: AST Parsing Failures

Solution: Syntax errors in source files crash `ast.parse()` — wrap all parsing in try/except and fall back to character chunking for malformed files; log all parse failures for debugging

Challenge: Chunk Size vs Context

Solution: AST chunks for very long functions (500+ lines) exceed LLM context windows — implement recursive chunk splitting: split large functions at logical sub-blocks while preserving metadata

Challenge: Embedding Cost

Solution: Embedding a large repo (1000+ files) can cost significantly via OpenAI APIs — use sentence-transformers (local, free) for development; switch to OpenAI embeddings only for production demo

Challenge: Retrieval Precision

Solution: Vector search retrieves semantically similar code but not always the code that answers the question — add BM25 for exact identifier matching and combine via RRF to improve precision

Challenge: Context Window Management

Solution: Top-K retrieved chunks may overflow LLM context — implement dynamic K selection based on token count; compress older conversation turns using summarisation

■ Learning Resources

- ◆ tree-sitter documentation — multi-language AST parsing
- ◆ ChromaDB documentation — local vector store

- ◆ LangChain RAG documentation — retrieval chain setup
 - ◆ RAGAS framework — RAG evaluation metrics
 - ◆ Anthropic API documentation — Claude integration
 - ◆ LlamaIndex documentation — code chunking modules
-

■ Project Completion Criteria

- GitHub repo cloner and file traverser functional for public repos
- Python AST chunker extracts all functions/classes with metadata
- tree-sitter chunker supports at least 2 additional languages (JS, Java)
- All chunks embedded and indexed in ChromaDB with metadata
- Hybrid retrieval (dense + BM25) implemented and validated
- Claude/GPT-4o integration answering questions with source citations
- Multi-turn conversation with sliding window memory functional
- FastAPI backend with all core endpoints live
- Streamlit chat UI with source chunk expandable references
- RAG evaluation showing AST chunking outperforms naive chunking on test set